Praxis: Sensor-Pipeline mit Message Queue

Geführte Implementierung mit vorbereitetem Projektgerüst

4.06

Datenfluss durch fünf Verantwortungsbereiche

Jede Stufe arbeitet gegen einen kleinen, klaren Vertrag

01	02	03	04	05
Emulator	**Erfassung**	**Probe**	**Message Queue**	**Auswertung**
deterministischer Stimulus	Sensor API + Validierung	Wert, Zeit, Folge, Qualität	kopiert vollständige Proben	letzter Wert + Zähler

Der Produzent kennt die Auswertung nicht; der Verbraucher kennt die Sensor API nicht.

Module im Projektgerüst

01	`sensor_capture.c`	plant Erfassung, liest Sensor und erzeugt motion_probe
02	`motion_probe.h`	definiert Struktur, Einheit, Qualität und gemeinsame Invarianten
03	`evaluator.c`	verarbeitet nur Proben, aktualisiert Zustand und Zähler

Nur das Erfassungsmodul kennt Device und Sensor API.

Produzent, Queue und Verbraucher

PRODUZENT

Periodisch erfassen

endliche Wartezeit

Sequenz setzen

Qualität bestimmen

MESSAGE QUEUE

Proben übertragen

feste Elementgröße

begrenzte Tiefe

explizite Vollstrategie

VERBRAUCHER

Fachlich auswerten

letzte gültige Probe

Datenalter berechnen

Zähler aktualisieren

Die Queue entkoppelt Ausführungskontexte – nicht die fachlichen Invarianten.

Startpunkte im Projektgerüst

DATEIEN UND FUNKTIONEN

```
sensor_capture.c → capture_probe()
motion_probe.h   → struct motion_probe
pipeline.c       → K_MSGQ_DEFINE(...)
evaluator.c      → consume_probe()
```

01 Erfassung

Vorhandene Sensorwerte in eine vollständige Probe überführen.

02 Übergabe

Probe mit festgelegter Wartezeit in die Queue einstellen.

03 Auswertung

Qualität prüfen, letzten gültigen Wert und Zähler aktualisieren.

Die Folie markiert Einstiegspunkte – die vollständige Implementierung liegt im Projektquellcode.

Message Queue als kleiner Vertrag

QUEUE-API

```
k_msgq_put(&probe_q, &probe, K_NO_WAIT);
k_msgq_get(&probe_q, &probe, K_MSEC(100));
```

01 Produzent

meldet Erfolg oder volle Queue und bleibt zeitlich begrenzt.

02 Verbraucher

wartet endlich, verarbeitet genau eine vollständige Probe.

Elementgröße, Tiefe und Wartezeiten sind Teil der überprüfbaren Architektur.

Wenn die Queue voll ist

ENTSCHEIDUNG

`Neueste bevorzugen · neue Probe verwerfen · Produzent begrenzen`

`Neueste bevorzugen`	älteste Probe entfernen, dann aktuellen Wert einstellen
`Neue Probe verwerfen`	Queue unverändert lassen und drop-Zähler erhöhen
`Produzent begrenzen`	mit endlicher Wartezeit auf freien Platz warten
`Projektentscheidung`	eine Strategie wählen und für alle Tests beibehalten
`Invariante`	kein stiller Verlust; jeder Drop wird gezählt

Queue-Voll ist ein erwartbarer Betriebszustand – kein unsichtbarer Sonderfall.

Zähler zeigen den Datenpfad

01	`samples_ok`	Probe erfasst und erfolgreich in die Queue eingestellt
02	`samples_dropped`	Probe wegen Queue-Strategie nicht weitergegeben
03	`samples_error`	Sensorzugriff oder Probe-Erzeugung fehlgeschlagen

Zähler werden an genau der Stelle erhöht, an der das Ereignis entschieden ist.

Qualitäts- und Zustandsinvarianten

AUSWERTUNGSZUSTAND

`last_valid · age_ms · samples_ok · samples_dropped · samples_error`

EINGANG	Auswertung akzeptiert ausschließlich struct motion_probe
LETZTER WERT	nur VALID ersetzt last_valid
DATENALTER	beim Verbraucher aus Uptime berechnen
SEQUENZLÜCKE	sichtbar machen und mit Drop-Zähler abgleichen

Der Anwendungszustand bleibt konsistent, auch wenn einzelne Proben fehlen oder fehlschlagen.

Abnahme: drei Stimulusfolgen

PIPELINE

```
Stimulus A → capture → msgq → evaluate
Stimulus B → capture → msgq → evaluate
Stimulus C → capture → msgq → evaluate
                          ↓
        last_valid · age_ms · counters
```

01 Nachweisen

Reihenfolge, Qualität und letzter gültiger Wert

02 Beobachten

Queue-Voll, Fehlerfall und erwartete Zählerdifferenz

Ergebnis: Die Sensor-Pipeline läuft unter native_sim reproduzierbar und bleibt modular.